

Ireen Cheralody

Toronto, ON | ireen.z.cheralody@gmail.com
linkedin.com/in/ireen-cheralody | github.com/ireen-z-cheralody

EDUCATION

University of Toronto

September 2024 - April 2029

Bachelor of Applied Science - **Computer Engineering + PEY Co-op**

AI Minor + Cybersecurity Certificate + Business Certificate

- **Relevant Courses:** Software Engineering, Introduction to Artificial Intelligence, Fundamentals of Deep Learning, Introduction to Databases, Algorithms & Data Structures, Operating Systems, Foundations and Frontiers of Cybersecurity

TECHNICAL SKILLS

Languages | Verilog, Assembly, C++, C, Python, Java, JavaScript, TypeScript, HTML, CSS

Software & Frameworks | React, Flask, PyTorch, NumPy, Librosa, GTK, Glade, Libcurl

Software Engineering | Algorithms, Data Structures, Multithreading, Optimization, APIs, Memory Debugging, Git/GitHub

Hardware Tools | Quartus Prime, ModelSim, DESim, CPULator, LTspice, Raspberry Pi

SOFTWARE & ML PROJECTS

Music Genre Categorization | Python, PyTorch, Librosa, NumPy, GitHub

May 2026 – August 2026

Solo Developer

Toronto, ON

- Developed a CNN music genre classifier using a dataset of **128-band mel spectrograms** from 30-second audio clips, achieving **79.46% test accuracy** across 7 genres
- Built an end-to-end audio preprocessing pipeline with **Librosa**, combining and balancing **GTZAN and FMA** into 4,625 samples with augmentation via time stretching, pitch shifting, and additive Gaussian noise
- Benchmarked the CNN against an SVM baseline (with **79.17% accuracy**) and evaluated against **35-70 unseen songs**, achieving **77.02% accuracy** on the final model

Tummi GIS | C/C++, CSS, HTML, EZGL, Libcurl, VNC, Glade, GTK, APIs, Git

January 2026 – April 2026

Co-Developer & Communication Liaison

Toronto, ON

- Developed a Geographical Information System aimed to easily find restaurants near you based on price, cuisine, and ratings.
- Engineered **Dijkstra + A*** pathfinding with turn-penalty handling, **reducing average route query time by ~40%** over baseline search
- Implemented a multithreaded **Traveling Courier Problem** solver using iterated local search, double-bridge perturbation, and or-opt moves across **4 parallel threads**, achieving solutions **within 15% of optimal**
- **Optimized** map rendering and routing through cached road geometry, background-threaded computation, and Valgrind debugging, **reducing per-frame render time by ~60%** while maintaining responsiveness on graphs with **200k+ edges**

UTwind Design Team | C, Python, Raspberry Pi

September 2025 – April 2026

Developer - Control Systems Sub-Team

Toronto, ON

- Developed a **Flask and React** application to monitor and visualize **real-time wind turbine control-system data**
- Built **REST APIs in Flask** to facilitate communication between the frontend and control-system backend, enabling responsive system monitoring
- Designed **interactive React components** to display turbine performance, operating conditions, and control data
- **Collaborated** with the Control Systems and Power Systems sub-teams to **integrate the application** with the turbine's hardware and software infrastructure

HARDWARE & EMBEDDED PROJECTS

Low-Level Garageband | C, Python, VGA Display, CPULator, Github

January 2026 – April 2026

Co-Developer

Toronto, ON

- Built a **GarageBand-style digital audio workstation in C (~3,000+ lines)** on a **DE1-SoC (RISC-V)**, integrating real-time audio, VGA graphics, and PS/2 keyboard/mouse input
- Implemented **memory-mapped audio playback** across multiple simultaneous channels and **double-buffered VGA** rendering, achieving **<10 ms audio latency and ~60 FPS** with no screen tearing

Low-Level Pacman | Verilog (HDL), Quartus, DESim, ModelSim

September 2025 – December 2025

Co-Developer

Toronto, ON

- Developed **~5,000 lines of Verilog** for an **FPGA-based Pac-Man** game with hierarchical FSMs for player movement, collision detection, scoring, and game-state control, integrating PS/2 keyboard input and VGA graphics at **~60 FPS**
- **Verified and debugged** the design using ModelSim and DESim, finding issues with VGA artifacts and maze boundaries, and **resolving hardware/simulation mismatches**